

Optimization Algorithms

Project Guidelines

Paulo Souza, Diogo Rasteiro & Liah Rosenfeld
April, 2026

1 Project Rules

This project is designed to assess your understanding of optimization methods and your ability to apply the algorithms developed throughout the practical classes to a real machine learning problem.

The objective is to use the optimization library you have been building over the course of the semester to train a neural network, replacing the standard learning procedure with two optimization algorithms: the Genetic Algorithm you have implemented, and a second algorithm of your choice from the nature-inspired and swarm intelligence family. You are expected to produce a working implementation of both, compare their performance against each other, and submit an accompanying report that describes your methodology and discusses your results.

Project groups may consist of three to four members. Team compositions must be submitted via Moodle by **May 8th**. The project submission deadline is **June 5th at 23:59**. Late submissions will be penalized according to the following criteria: projects submitted up to one day after the deadline will incur a deduction of 2 points, those submitted two days late will lose 5 points, and those submitted three days late will lose 10 points. Any project delivered more than three days past the deadline will automatically receive a grade of 0 points.

All projects must be presented and defended on the week from **June 8th to June 12th**. The specific days and time for each defense will be communicated to you closer to this date, as it depends on the number of registered groups. During the defense, you will be required to explain your implementation, justify your design choices, and discuss your results. Each student must be familiar with the entire project. Inadequate answers may result in grade deductions, and failure to attend the defense will result in a grade of zero for the project.

2 Project Description

Artificial neural networks are parametric models that learn to map inputs to outputs by adjusting a large number of internal parameters, commonly referred to as *weights*. In standard practice, these weights are estimated using gradient-based methods that iteratively update the parameters in the direction that reduces prediction error.

However, gradient-based methods are not the only way to find good weights for a neural network. Optimization algorithms of the kind you have been studying throughout this course can also be used to search the space of possible weight configurations and find those that lead to good predictive performance.

This project asks you to do exactly that: treat the weights of a neural network as the *solution* to an optimization problem, and use your library to search for a configuration that minimizes prediction error on a dataset. Concretely, you will implement and apply two optimization algorithms to this problem, compare their performance, and critically discuss your findings.

Please note that the primary focus of the evaluation will be the *process* you use, and not the *result*. Groups that apply the theoretical concepts learned in class, use their creativity and are both rigorous and exhaustive with their testing will be regarded more favourably. Nevertheless, consider that an excellent methodology is often highly correlated with robust results.

2.1 The Optimization Problem

Consider a neural network with a fixed architecture (i.e., a fixed number of layers and neurons per layer). Once the architecture is decided, the network's behavior is entirely determined by its weights. A solution to

the optimization problem is therefore a vector:

$$\theta = (w_1, w_2, \dots, w_n) \in \mathbb{R}^n$$

where n is the total number of weights in the network and each w_i is a real-valued scalar. The objective is to maximize the predictive performance of the network. The choice of how to measure performance is left to you and should be justified in your report.

This formulation is compatible with both algorithms you will implement. The role of `generate_solution` is to produce a random weight vector and the role of `fitness_function` is to evaluate how well a given weight configuration performs on the dataset. Both algorithms share these components and can therefore be compared directly and fairly.

2.2 Representing the Neural Network

For this project, you are expected to use the `scikit-learn` library to instantiate and evaluate neural networks. Specifically, you should use the `MLPClassifier` class, which allows you to set the network's weights externally. You can do this with attributes such as `coefs_`, keeping in mind the difference in shape. For more information regarding how to accomplish this, you are recommended (and expected to read) the official documentation available [here](#).

The dataset for this project is the **Oxford Parkinson's Disease Detection dataset**, whose information can be found [here](#), which will be provided to you via Moodle in a preprocessed form. The dataset consists of a set of biomedical voice measurements collected from individuals, and the task is to distinguish between healthy subjects and those with Parkinson's disease based on these measurements. It is provided already standardised, as neural network weights are sensitive to the scale of the input features and working with raw data would make the optimization significantly harder for reasons unrelated to the algorithms themselves. You should not need to perform additional preprocessing, unless you find it relevant to do so.

3 Project Requirements

The following section describes the requirements your project must fulfil to be considered successful. Any additional functionality beyond what is described here is encouraged, provided it does not conflict with any stated requirement.

3.1 Core Implementation

Your implementation must include the following components, organized in a clear and consistent manner within your codebase.

Solution Representation

Define a function `generate_solution` that produces a random weight vector of the appropriate length for your chosen network architecture. The length of this vector must match the total number of weights and biases in the network, which depends on the number of layers and neurons you choose. If you decide to include further parameters in your optimization, as suggested in the **Optional Extensions** section, you need to elaborate a plan to include those in the representation of your solution.

Fitness Function

Define a function `fitness_function` that, given a weight vector, loads those weights into the network, runs predictions on the dataset, and returns a value reflecting the predictive performance of that configuration. The choice of performance metric is yours to make and must be justified in your report. Consider what it means to perform well on this particular dataset and whether a single metric fully captures that.

Genetic Operators

Define crossover and mutation operators compatible with the Genetic Algorithm, adapted to operate on real-valued vectors rather than binary ones. Since the weight vector is continuous, both operators will need to differ from those used in the Knapsack problem; you should think carefully about what it means to combine or perturb real-valued solutions meaningfully.

For a complete project, you are required to implement the following operators, beyond those implemented in class:

- 2 Crossover operators.
- 2 Mutation operators.
- 1 Selection operator.
- 2 separate initialization methods.

Your report should contain a comparison between them, as well as the operators developed in class.

Second Algorithm — Nature-Inspired / Swarm Algorithms

In addition to the Genetic Algorithm, you must implement one algorithm of your choice from the following family of nature-inspired and swarm intelligence methods:

- Particle Swarm Optimization (PSO)
- Artificial Bee Colony (ABC)
- Grey Wolf Optimizer (GWO)
- Whale Optimization Algorithm (WOA)
- Differential Evolution (DE)
- Firefly Algorithm
- Bat Algorithm
- Harmony Search
- Cuckoo Search
- Sine Cosine Algorithm

Your implementation must be integrated into your library following the same conventions as the other algorithms, so that it can be applied to the same problem using the same `fitness_function`. Both algorithms must be your own implementations and the use of external optimization libraries is not permitted. You are expected to research the algorithm you choose and justify your choice in the report. While the implementation details are left to you, all members of the group must be able to explain it in the defense, if asked.

Experimental Evaluation

Run both algorithms on the dataset and record the results. How you assess your models and present your findings is for you to decide and no specific format or evaluation procedure is imposed. What will be evaluated is the quality and rigour of what you report: whether your results are statistically sound, whether you have explored each algorithm's parameters meaningfully, and whether your conclusions about the comparison between the two methods are well supported by the evidence you present.

3.2 Optional Extensions

Groups that wish to go beyond the core requirements are encouraged to explore additional extensions to their work. They are not required but will be taken into account positively during evaluation. Some examples include:

- Optimizing the layout and configuration of the neural network.
- Optimizing the other parameters of the `MLPClassifier`.
- Applying additional Machine Learning concepts in order to improve performance of the network.
- Experimenting with additional algorithms of the Nature-Inspired family.
- Experimenting with additional operators for GAs beyond the required ones.
- Using statistical tests to support conclusions drawn from the comparison between algorithms.
- Any other novel ideas you may have.

No rigid structure will be imposed for these additional extensions. You are encouraged to use your creativity and attempt to push your model's performance as high as possible.

3.3 Report

You are required to submit a written report together with your code. The report should be concise and well-structured, and must include the following sections.

Problem Formulation

A clear description of the optimization problem you defined, including the solution representation, the fitness function, and the parameter configurations you settled on for each algorithm. For the second algorithm, additionally explain your choice and provide a concise description of how it works.

Implementation Details

A description of the key functions you implemented and how they connect your optimization library to the neural network. Cover both algorithms and highlight any meaningful differences in how they explore the search space. You do not need to reproduce your entire code in the report, but you should describe the most important design decisions.

Experimental Results

A presentation of your results for both algorithms, allowing for a direct comparison between them. Results must be averaged over multiple runs to ensure statistical robustness.

Discussion

A critical analysis of your results. How do the two algorithms compare in terms of performance and convergence? How did your parameter choices influence the results, and what would you change with more time? What are the main limitations of using population-based optimization algorithms to train a neural network?

Instructions

Clear and concise instructions on how to run your code, including any dependencies that need to be installed.

4 Delivery

All groups should deliver their projects by **June 5th** at **23:59**, with any delays resulting in deductions according to the rules stated in Section 1.

Your delivery should consist of the following items, contained within a single zip file:

- A folder containing all Python files necessary to run your project, including your library files and any additional scripts you wrote for this project.
- A PDF copy of your report.

This zip file should be named **Group_XX.zip**, where **XX** represents your group number as assigned on Moodle. Only one member per group needs to submit. As a reminder, after the deadline your submission is considered final and can no longer be altered, so please ensure that you submit the correct version and that everything is functioning properly before you do so.

After the submission, you will be required to attend a defense session. During it, you will be asked questions regarding your implementation, your results, and the theoretical concepts that underlie your work. Depending on your answers, your grade may be adjusted. Failure to be present will result in a grade of 0 for the project.